

Multimodal Deep Learning

Answer Sheet 7

Date: July 1, 2025

Exercise 1: Selective Fine-Tuning

(a) Fishmask

- (i) Write the formula that maps the vector of scores $\mathbf{I}$ to a binary mask $m \in \{0, 1\}^n$ respecting the budget constraint.

Problem Statement:

- A layer contains 4.0×10^6 weights.
- “Fisher score approximation” assigns a scalar importance I_i to each weight w_i .
- We are allowed to update only 0.8% of these weights.
- This refers to the Fisher Information Matrix (FIM), which measures how sensitive the loss function is to parameter changes.
 - High Fisher score I_i : important weight.
 - Low Fisher score I_i : less important.
 - One-shot Fisher approximation computes importance without retraining.

Answer:

- Total weights: 4 million parameters.
 - Budget: 0.8% of total weights = 32,000 weights = k .
 - A binary mask $\mathbf{m} \in \{0, 1\}^n$ has 32,000 ones and the rest zeros. ($m_i = 1$: update; $m_i = 0$: freeze).
 - Mapping formula (using both $\mathbf{I}$ and $\mathbf{m}$): $\mathbf{M} = \mathbf{I}^T \mathbf{m}$.
 - Mapping formula (constructing $\mathbf{m}$ from top- k): $\mathbf{m} = \mathbf{I} > \min(\text{Top}_k(\mathbf{I}))$.
- (ii) To update the selected weights, the Adam optimizer requires memory for more than just the weights themselves. For each trainable parameter, memory must be allocated for its **gradient**, its **1st moment**, and its **2nd moment**. Assuming all values are stored in 32-bit float precision (4 bytes per number), what is the total memory overhead for the optimizer?

Answer:

- Total memory for weights: 0.8% of 4.0×10^6 weights $\times$ 3 stages $\times$ 4 bytes = 375 MB.

(b) DiffPruning

- (i) Compute the GPU memory overhead (in MB) of storing the dense differentiable mask during training.

Problem Statement:

- A model of 350 M-parameter.
- Aiming to prune or zero out 99.5% of parameters – leaving only 0.5% of the weights active. This is one of many methods to compress a model which lessen memory used and improve generalization (reduce overfitting).

- The dense differentiable mask is a tensor of the same size as the model's parameters, with values between 0 and 1.

Answer: Total memory for the *dense* differentiable mask: $350 \times 10^6 \times \text{size of FP32}$ (4 bytes) ≈ 1335.14 MB.

- (ii) If you could store the final mask as a packed 1-bit tensor for inference, what would its size be (in kB)?

Answer: Total memory for the final mask: $350 \times 10^6 \times 1 \text{ bit} = 42,675.78$ kB.

(c) **BitFit**

Note: BitFit is a lightweight fine-tuning method where only bias terms are updated during training. All other weights (e.g., attention layers, MLPs, etc.) are frozen.

A GPT-style model has 40 transformer blocks with the following per-block sizes:

- Query bias: $d_{\text{model}} = 768$
- Middle-MLP bias: $d_{\text{ff}} = 3072$

- (i) Calculate the percentage of the model's 250 M parameters that BitFit updates.

Answer: A single transformer block contains $768 + 3072 = 3840$ parameters (bias weights). For 40 transformer blocks, there are $40 \times 3840 = 153,600$ updated by BitFit, equivalent to 0.0614% of a model with 250 M parameters.

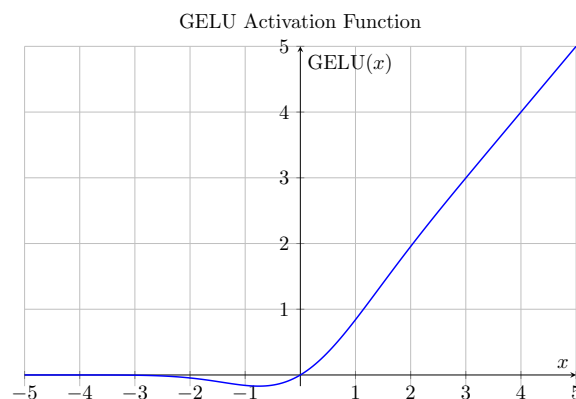
- (ii) State one reason why BitFit's advantage fades on very large models or on architectures like LLaMA.

Answer: Bias terms make up a very small fraction of the total parameters, so updating only those (as BitFit does) has limited influence on the model's behavior.

Exercise 2: Adapter

- (a) You are adding adapter modules to a transformer with a hidden dimension of $d_{\text{model}} = 768$. Each adapter is a bottleneck feed-forward network with a reduction factor, such that the inner dimension is $r = 16$. The adapter structure is:

$$\text{Linear}(d_{\text{model}} \rightarrow r) \rightarrow \text{GELU} \rightarrow \text{Linear}(r \rightarrow d_{\text{model}})$$



- (i) Calculate the number of trainable parameters in a **single** adapter module. Remember to include bias terms for both linear layers.

Answer:

The number of weights:

$$\begin{aligned}\#w_i &= (768 \times 16) + (16 \times 768) \\ \therefore \#w_i &= 24576\end{aligned}$$

The number of biases:

$$\begin{aligned}\#b_i &= 16 + 768 \\ \therefore \#b_i &= 784\end{aligned}$$

Total parameters = 25360 parameters

- (ii) Explain why this method, despite its parameter efficiency, significantly increases inference latency compared to a method like (IA)³.

Answer: First, GeLU preserves small input signals, unlike ReLU. Also, a (sequential/parallel) adapter allows its parameters to be fully updated, and might help generalization.

(b) (IA)³

You are fine-tuning the same 40-block, 250 M-parameter model from the BitFit exercise using (IA)³. This method introduces three learnable scaling vectors per transformer block:

- $\mathbf{l}_v$ and $\mathbf{l}_k$ to scale the value and key activations, each a learnable vector of length $d_{\text{model}} = 768$.
- $\mathbf{l}_{\text{ffn}}$ to scale the output of the first MLP layer, a learnable vector of length $d_{\text{ff}} = 3072$.

- (i) Calculate the total number of trainable parameters introduced by (IA)³ for the entire 40-block model.

Answer: A single block of (IA)³ contains $\mathbf{l}_v$, $\mathbf{l}_k$, $\mathbf{l}_{\text{ffn}}$ as learnable vectors of length 768, 768 and 3072 (Total of 4608 parameters). Hence, with 40-block model, $4608 \times 40 = 184320$ parameters.

- (ii) Based on its mechanism, why is (IA)³ considered to have more limited expressibility than a standard adapter module?

Answer: (IA)³ updates only the scaling of attention and MLP. It does not produce linear projection or nonlinear transformation, like standard adapters.